P2299R3
`mdspan` of All Dynamic Extents

Published Proposal, 2021-06-08

Author:

Bryce Adelstein Lelbach (he/him/his) — Library Evolution Chair (NVIDIA)

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

§ 1. Introduction

[P0009R12] proposes adding non-owning multidimensional span abstractions to the C++ Standard Library; `basic_mdspan`, which is fully generic and can represent any kind of multidimensional data, and `mdspan`, a convenience template alias for simpler use cases.

```
template <class ElementType, class Extents, class LayoutPolicy = layout_righ
          class AccessorPolicy = accessor_basic<ElementType>>
struct basic_mdspan;

template <class ElementType, size_t... Extents>
using mdspan = basic_mdspan<ElementType, extents<Extents...>>;
```

In the `basic_mdspan`/`span` interface, extents can be either static, e.g. expressed at compile time:

```
mdspan<double, 64, 64> a(data);
```

or dynamic, e.g. expressed at run time:

```
mdspan<double, dynamic_extent, dynamic_extent> a(data, 64, 64);
```

You can also use a mix of the two styles:

```
mdspan<double, 64, dynamic_extent> a(data, 64);
```

While `basic_mdspan` and `mdspan` are powerful, the spelling of instantiations can be verbose, especially for the common case where all extents are dynamic.

§ 2. Class Template Argument Deduction for All Dynamic Extents

Using class template argument deduction (introduced in C++17) and alias template argument deduction (introduced in C++20), we can make it a easier to use `mdspan` with all dynamic extents:

Before	After
`mdspan<double, dynamic_extent, dynamic_extent> a(data, 64, 64);`	`mdspan a(data, 64, 64);`

To make this work, we need to add a deduction guide for `basic_mdspan`. Through the power of alias template argument deduction, `mdspan` will be able to use said deduction guide as well.

Here's an example implementation of such a deduction guide for `basic_mdspan`.

In earlier versions of this paper, it was unclear whether such a deduction guide would work for the alias template `mdspan`, as attempts to construct one that functioned as intended had failed. However, we have since learned that the this is due to bugs in the two implementations that currently support alias template deduction, MSVC and GCC. Those bugs have been reported to the respective compilers and hopefully will be fixed soon.

Here's an example of the current implementation bugs that prevent the deduction guide from working for `mdspan`.

3. Template Aliases for All Dynamic Extents

The root of the interface challenge here is the heterogeneous nature of extents in `mdspan`. When you need to express static extents or a mix of static and dynamic extents, we must state the kind of every extent, similar to how we state the type of every element of a `tuple`. This heterogenity gives us flexibility at the cost of verbosity.

However, in the case of all dynamic extents, we do not need that flexibility; the kind of every extent is the same. Instead of a `tuple`-like interface, we likely want an `array`-like interface, where we can simple state the number of extents, instead of listing every one.

We can provide this simplified interface easily by adding a template alias for `extents` that takes the number of extents as a parameter and expands to an `extents` of that many `dynamic_extents`. Here's an example implementation.

tuple-like	**array-like**
`tuple<T, T, T>`	`array<T, 3>`
`extents<dynamic_extent, dynamic_extent, dynamic_extent>`	`dextents<3>` **(proposed)**

If you are working with an `array`-like representation of extents, working with `mdspan` can become quite awkward:

```
template <size_t>
constexpr auto make_dynamic_extent() {
  return dynamic_extent;
}

template <typename T, size_t Rank>
struct tensor {
  T* data;
  array<size_t, Rank> extents;
```

```
  auto get_mdspan() {
    return get_mdspan_impl(make_index_sequence<Rank>());
  }

  template <size_t... Pack>
  auto get_mdspan_impl(index_sequence<Pack...>) {
    return mdspan<T, make_dynamic_extent<Pack>()...>(data, extents);
  }
};
```

With `dextents`, this becomes a lot simpler:

```
template <typename T, size_t Rank>
struct tensor {
  T* data;
  array<size_t, Rank> extents;

  template <size_t... Pack>
  auto get_mdspan_impl(index_sequence<Pack...>) {

  }
};
```

§ 4. Remove `mdspan` Convenience Alias

`dextents` will have to be used with the unfortunately named `basic_mdspan`. I considered proposing something along the lines of `dynamic_mdspan<T, Rank>`. However, this would potentially lead to confusion, as a `size_t` non-type template parameter in `mdspan` would mean "static extent of this size" while a `size_t` non-type template parameter in `dynamic_mdspan` would mean "number of extents".

Syntax	Semantics
`span<T, N>`	`span` with a static size of`N`.
`mdspan<T, N>`	`mdspan` with a single static extent of size `N`.
`dynamic_mdspan<T, N>` **(not proposed)**	`mdspan` with `N` dynamic extents.

Instead, I think we should:

— Delete the `mdspan<T, Extents...>` alias.

— Rename `basic_mdspan` to `mdspan`.

The purpose of the `mdspan` alias was to provide a simple and easy to use interface for those with less complex and advanced use cases. When the `mdspan`/`basic_mdspan` dichotomy was introduced when the proposal was first under consideration, before the existence of Class Template Argument Deduction and the proposed `dextents`. With the combination of these two forces, I believe that a separate `mdspan` alias and `basic_mdspan` class are unnecessary to achieve a simple and easy to use interface.

Before	`mdspan<T> m(data, 16, 64, 64);`
After	`mdspan<T> m(data, 16, 64, 64);`

Before	`mdspan<T, dynamic_extent, dynamic_extent, dynamic_extent> f();`
After	`mdspan<T, dextents<3>> f();`

Before	`mdspan<T, 3, 3> m;`
After	`mdspan<T, extents<3, 3>> m;`

Before	`mdspan<T, 3, 3> f();`
After	`mdspan<T, extents<3, 3>> f();`

Before	`mdspan<T, 16, dynamic_extent, 64> m;`
After	`mdspan<T, extents<16, dynamic_extent, 64>> m;`

Before	`mdspan<T, 16, dynamic_extent, 64> f();`
After	`mdspan<T, extents<16, dynamic_extent, 64>> f();`

§ 5. Wording

The following changes are relative to the `mdspan` proposal ([P0009R12]).

The `[]` character is used to denote a placeholder number which shall be selected by the editor.

Modify the header synopsis for `<mdspan>` in [**mdspan.syn**] as follows:

22.7.[] Header <mdspan> synopsis [mdspan.syn]

```
namespace std {
  // [mdspan.extents], class template extents
  template
    class extents;

  template<size_t Rank>
    using dextents = decltype([] <size_t... Pack> (index_sequence<Pack...
                        return extents<[] (auto) constexpr
                                       { return dynamic_extent; }
                                       (integral_constant<size_t, Pack>{})
                      }(make_index_sequence<Rank>{}));

  // [mdspan.layout], Layout mapping policies
  class layout_left;
  class layout_right;
  class layout_stride;

  // [mdspan.accessor.basic]
  template<class ElementType>
    class default_accessor;

  // [mdspan.basic], class template mdspan
  template<class ElementType, class Extents, class LayoutPolicy = layout_
           class AccessorPolicy = default_accessor<ElementType>>
    class basic_mdspan;

  template <class ElementType, class... Integrals>
  explicit basic_mdspan(ElementType*, Integrals...)
    -> basic_mdspan<ElementType, dextents<sizeof...(Integrals)>

  template<class T, size_t... Extents>
    using mdspan = basic_mdspan<T, extents<Extents...>>;

  // [mdspan.submdspan]
  template<class ElementType, class Extents, class LayoutPolicy,
           class AccessorPolicy, class... SliceSpecifiers>
    constexpr basic_mdspan<see below>
    submdspan(const basic_mdspan<ElementType, Extents, LayoutPolicy, Acce
              SliceSpecifiers... specs) noexcept;

  // tag supporting submdspan
```

```
  struct full_extent_t { explicit full_extent_t() = default; };
  inline constexpr full_extent_t full_extent = full_extent_t{};
}
```

Replace all occurences of `basic_mdspan` with `mdspan`.

References

Informative References

[P0009R12]

Christian Trott, D.S. Hollman, Damien Lebrun-Grandie, Mark Hoemmen, Daniel Sunderland, H. Carter Edwards, Bryce Adelstein Lelbach, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Nevin Liber. MDSPAN. 21 May 2021. URL: https://wg21.link/p0009r12